

Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch

```
oraiz@LAPTOP-SOMNC04S MINGW64 ~/Desktop/u1ayjowonbkup/New folder/Gayl/FLUTTER/IT314AB_ORAIZ (feature/data-ui)
$ git checkout -b feature/student-list
Switched to a new branch 'feature/student-list'

oraiz@LAPTOP-SOMNC04S MINGW64 ~/Desktop/u1ayjowonbkup/New folder/Gayl/FLUTTER/IT314AB_ORAIZ (feature/student-list)
$
```

🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart `List`.

Research what a Dart `List` is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

Screenshot:

Code:

- Your student list
- At least five student entries

```
class Student {  
  String images;  
  String name;  
  String course;  
  int yearLevel;  
  int age;  
  String hobby;  
  
  Student({  
    this.images = '',  
    this.name = '',  
    this.course = '',  
    this.yearLevel=0,  
    this.age=0,  
    this.hobby= '',  
  })  
};
```

```
List<Student> students = [  
  Student(  
    name: '👩 Cassandra Gayle R. Oraiz👩',  
    age: 20,  
    course: 'BSIT',  
    yearLevel: 3,  
    hobby: 'Drawing, Gaming, Crochet',  
    images: 'assets/Mc.jpg'  
  ),  
  
  Student(  
    name: '🍪 Pure Vanilla Cookie🍪',  
    age: 20,  
    course: 'BNA',  
    yearLevel: 4,  
    hobby: 'Gardening',  
    images: 'assets/PV.jpg'  
  ),  
  
  Student(  
    name: '👱 Rin Itoshi👱',  
    age: 22,  
    course: 'BSS',  
    yearLevel: 3,  
    hobby: 'Horror enthusiast',  
    images: 'assets/67.jpg',  
  ),  
  
  Student(  
    name: '✨ Theodosia "Ran" Agapov ✨',  
    age: 23,  
    course: 'BCS',  
    yearLevel: 3,  
    hobby: 'Football, Hairstyling',  
    images: 'assets/Ran.jpg'  
  ),  
  
  Student(  
    name: 'Shadow Milk Cookie',  
    age: 25,  
    course: 'BFA',  
    yearLevel: 4,  
    hobby: 'Chismis',  
    images: 'assets/SMC.jpg'  
  )  
];
```

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)

```
Card(
  elevation: 4,
  shape: RoundedRectangleBorder(
    side: BorderSide(color: Colors.redAccent, width: 1),
    borderRadius: BorderRadius.circular(12),
  ), // RoundedRectangleBorder
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      SizedBox(height: 5),
      Text(
        students[0].name,
        style: TextStyle(
          fontSize: 25.0,
          color: Colors.teal.shade900,
          fontWeight: FontWeight.bold,
        ), // TextStyle
      ), // Text
      Row(
        mainAxisAlignment: .center,
        children: [
          Text(
            students[0].course,
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
          Text(
            students[0].yearLevel.toString(),
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
        ], // Row
      ), // Row
      Row(
        mainAxisAlignment: .center,
        children: [Text(students[0].age.toString())],
      ), // Row
      RainbowText(
        students[0].hobby,
        colors: [Colors.red, Colors.blue, Colors.green],
        style: TextStyle(fontStyle: FontStyle.italic),
      ), // RainbowText
      SizedBox(height: 5),
      Image.asset(students[0].images, height: 150, width: 150),
    ], // Column
  ), // Card
```

```
Card(
  elevation: 4,
  shape: RoundedRectangleBorder(
    side: BorderSide(color: Colors.redAccent, width: 1),
    borderRadius: BorderRadius.circular(12),
  ), // RoundedRectangleBorder
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      SizedBox(height: 5),
      Text(
        students[1].name,
        style: TextStyle(
          fontSize: 25.0,
          color: Colors.teal.shade900,
          fontWeight: FontWeight.bold,
        ), // TextStyle
      ), // Text
      Row(
        mainAxisAlignment: .center,
        children: [
          Text(
            students[1].course,
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
          Text(
            students[1].yearLevel.toString(),
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
        ], // Row
      ), // Row
      Row(
        mainAxisAlignment: .center,
        children: [Text(students[1].age.toString())],
      ), // Row
      RainbowText(
        students[1].hobby,
        colors: [Colors.red, Colors.blue, Colors.green],
        style: TextStyle(fontStyle: FontStyle.italic),
      ), // RainbowText
      SizedBox(height: 5),
      Image.asset(students[1].images, height: 150, width: 150),
    ], // Column
  ), // Card
```

```
Card(
  elevation: 4,
  shape: RoundedRectangleBorder(
    side: BorderSide(color: Colors.redAccent, width: 1),
    borderRadius: BorderRadius.circular(12),
  ), // RoundedRectangleBorder
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      SizedBox(height: 5),
      Text(
        students[2].name,
        style: TextStyle(
          fontSize: 25.0,
          color: Colors.teal.shade900,
          fontWeight: FontWeight.bold,
        ), // TextStyle
      ), // Text
      Row(
        mainAxisAlignment: .center,
        children: [
          Text(
            students[2].course,
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
          Text(
            students[2].yearLevel.toString(),
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
        ], // Row
      ), // Row
      Row(
        mainAxisAlignment: .center,
        children: [Text(students[2].age.toString())],
      ), // Row
      RainbowText(
        students[2].hobby,
        colors: [Colors.red, Colors.blue, Colors.green],
        style: TextStyle(fontStyle: FontStyle.italic),
      ), // RainbowText
      SizedBox(height: 5),
      Image.asset(students[2].images, height: 150, width: 150),
    ], // Column
  ), // Card
```



```

Card(
  elevation: 4,
  shape: RoundedRectangleBorder(
    side: BorderSide(color: Colors.redAccent, width: 1),
    borderRadius: BorderRadius.circular(12),
  ), // RoundedRectangleBorder
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      SizedBox(height: 5),
      Text(
        students[3].name,
        style: TextStyle(
          fontSize: 25.0,
          color: Colors.teal.shade900,
          fontWeight: FontWeight.bold,
        ), // TextStyle
      ), // Text

      Row(
        mainAxisAlignment: .center,
        children: [
          Text(
            students[3].course,
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
          Text(
            students[3].yearLevel.toString(),
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
        ],
      ), // Row

      Row(
        mainAxisAlignment: .center,
        children: [Text(students[3].age.toString())],
      ), // Row
      RainbowText(
        students[3].hobby,
        colors: [Colors.red, Colors.blue, Colors.green],
        style: TextStyle(fontStyle: FontStyle.italic),
      ), // RainbowText
      SizedBox(height: 5),

      Image.asset(students[3].images, height: 150, width: 150),
    ], // Column
  ), // Card

```

```

Card(
  elevation: 4,
  shape: RoundedRectangleBorder(
    side: BorderSide(color: Colors.redAccent, width: 1),
    borderRadius: BorderRadius.circular(12),
  ), // RoundedRectangleBorder
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      SizedBox(height: 5),
      Text(
        students[4].name,
        style: TextStyle(
          fontSize: 25.0,
          color: Colors.teal.shade900,
          fontWeight: FontWeight.bold,
        ), // TextStyle
      ), // Text

      Row(
        mainAxisAlignment: .center,
        children: [
          Text(
            students[4].course,
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
          Text(
            students[4].yearLevel.toString(),
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 19,
            ), // TextStyle
          ), // Text
        ],
      ), // Row

      Row(
        mainAxisAlignment: .center,
        children: [Text(students[4].age.toString())],
      ), // Row
      RainbowText(
        students[4].hobby,
        colors: [Colors.red, Colors.blue, Colors.green],
        style: TextStyle(fontStyle: FontStyle.italic),
      ), // RainbowText
      SizedBox(height: 5),

      Image.asset(students[4].images, height: 150, width: 150),
    ], // Column
  ), // Card

```

🚩 Flag 3 — Discover **ListView**

Objective: Make your student directory scrollable using **ListView**.

Research Flutter's **ListView**.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

Test

The user should be able to:

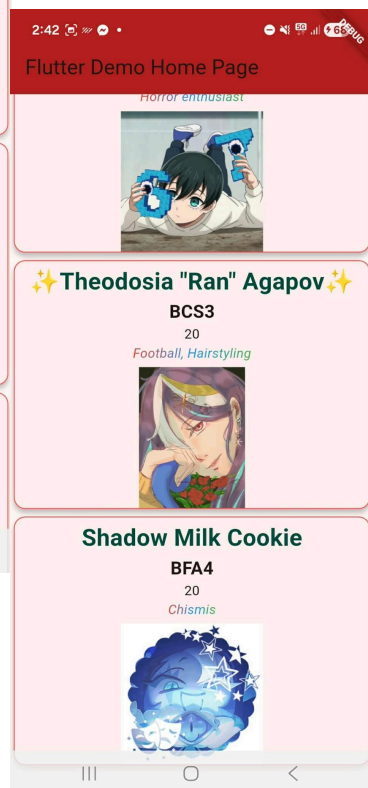
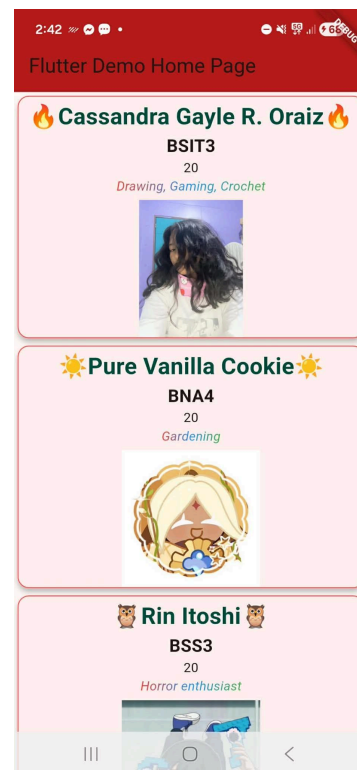
- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:

- **ListView** in your code
- Top of the application
- Bottom portion showing later students

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Colors.red[900],

      title: Text(widget.title),
    ), // AppBar
    body: ListView(
      children: [
        weBuildThings(students[0]),
        weBuildThings(students[1]),
        weBuildThings(students[2]),
        weBuildThings(students[3]),
        weBuildThings(students[4]),
      ],
    ), // ListView
  ); // Scaffold
}
```



🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

Test

Add one new student to your list.

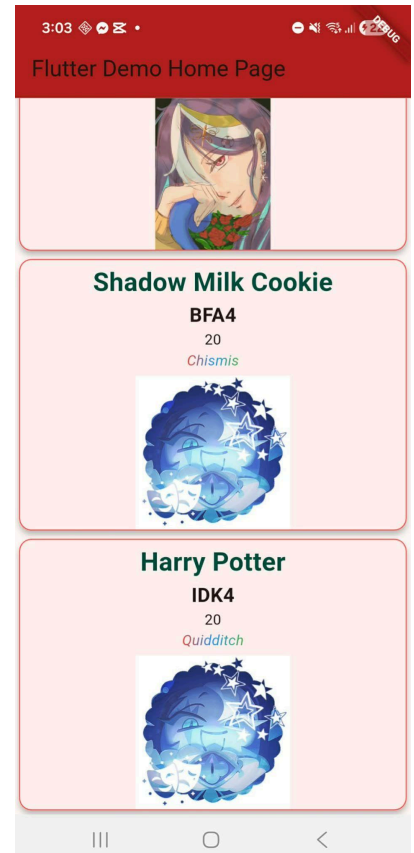
Without creating another card manually, your application should display the new student automatically.

Screenshot:

- `ListView.builder` in your code
- Student list
- Newly generated student appearing in the app

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Colors.red[900],
      title: Text(widget.title),
    ), // AppBar
    body: ListView.builder(
      itemCount: students.length,
      itemBuilder: (context, index) {
        return weBuildThings(students[index]);
      },
    ), // ListView.builder
  ); // Scaffold
}
```

```
),
Student(
  name: 'Shadow Milk Cookie',
  age: 25,
  course: 'BFA',
  yearLevel: 4,
  hobby: 'Chismis',
  images: 'assets/SMC.jpg',
),
Student(
  name: 'Harry Potter',
  age: 25,
  course: 'IDK',
  yearLevel: 4,
  hobby: 'Quidditch',
  images: 'assets/SMC.jpg',
),
];
```



🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

Examples:

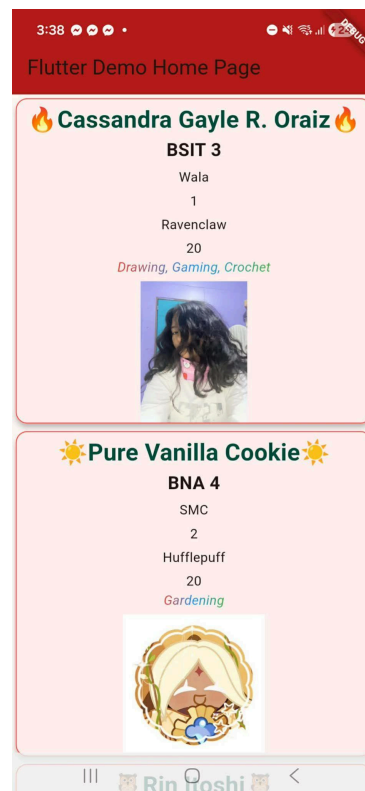
- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

Screenshot:

- Updated student data
- Student Card displaying the additional fields

```
class Student {  
  String images;  
  String name;  
  String course;  
  int yearLevel;  
  int age;  
  String hobby;  
  int ID;  
  String likes;  
  String house;  
  
  Student({  
    this.images = '',  
    this.name = '',  
    this.course = '',  
    this.yearLevel = 0,  
    this.age = 0,  
    this.hobby = '',  
    this.ID = 0,  
    this.likes = '',  
    this.house = '',  
  });  
}
```



🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

- "No students found."
- "Student list is empty."

Test

- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

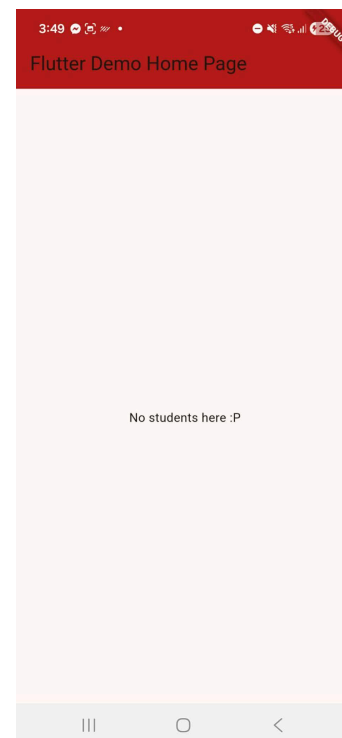
Screenshot:

- Empty student list
- Empty-state UI
- Application still functioning correctly

```
List<Student> students = [];
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Colors.red[900],

      title: Text(widget.title),
    ), // AppBar
    body: students.isEmpty
      ? Center(child: Text('No students here :P'))
      : ListView.builder(
          itemCount: students.length,
          itemBuilder: (context, index) {
            return weBuildThings(students[index]);
          },
        ), // ListView.builder
  ); // Scaffold
}
```



🚩 Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a `List`.

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

After sorting:

- Aaron
- John
- Maria
- Sophia

Your UI should immediately reflect the new order.

Screenshot:

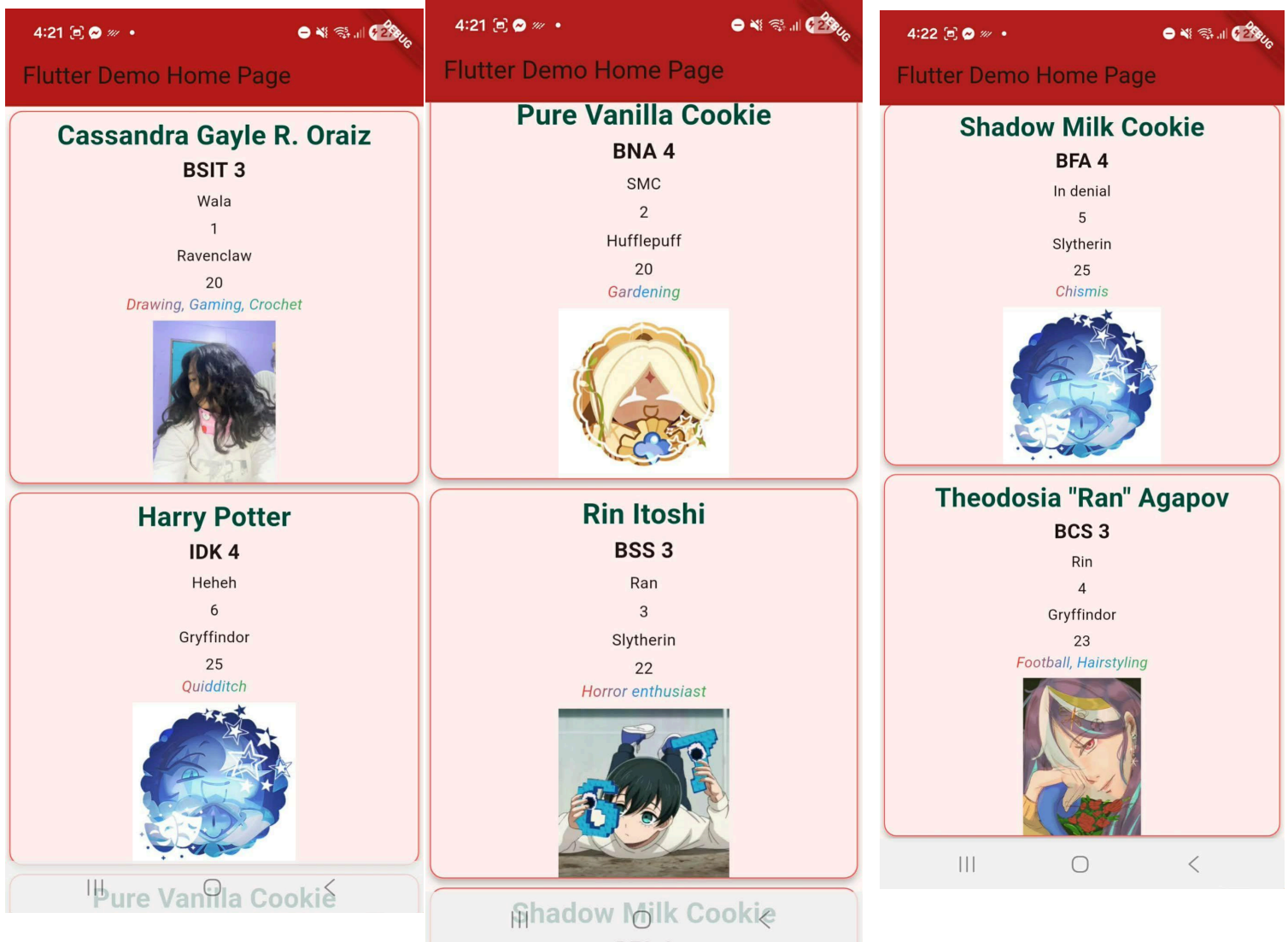
- Student list before or after sorting
- Application displaying the sorted order

//Put the command in void main. Won't work if not.

```
Run | Debug | Profile
void main() {
  runApp(const MyApp());
  students.sort((a, b) => a.name.compareTo(b.name));
}
```

```
List<Student> students = [
  Student(
    ID: 1,
    house: 'Ravenclaw',
    likes: 'Mala',
    name: 'Cassandra Gayle E. Orisz',
    age: 20,
    course: 'BSIT',
    yearLevel: 3,
    hobby: 'Drawing, Gaming, Crochet',
    images: 'assets/Mo.jpg',
  ),
  Student(
    ID: 2,
    house: 'Hufflepuff',
    likes: 'SMC',
    name: 'Pure Vanilla Cookie',
    age: 20,
    course: 'BMA',
    yearLevel: 4,
    hobby: 'Gardening',
    images: 'assets/PV.jpg',
  ),
  Student(
    ID: 3,
    house: 'Slytherin',
    likes: 'Rin',
    name: 'Rin Itoshi',
    age: 22,
    course: 'BS',
    yearLevel: 3,
    hobby: 'Horror enthusiast',
    images: 'assets/IT.jpg',
  ),
  Student(
    ID: 4,
    house: 'Gryffindor',
    likes: 'Rin',
    name: 'Theodora "Tan" Agapov',
    age: 23,
    course: 'BES',
    yearLevel: 3,
    hobby: 'Football, Hairstyling',
    images: 'assets/Ran.jpg',
  ),
  Student(
    ID: 5,
    house: 'Slytherin',
    likes: 'In denial',
    name: 'Shadow Milk Cookie',
    age: 25,
    course: 'SFA',
    yearLevel: 4,
    hobby: 'Chicoris',
    images: 'assets/SMC.jpg',
  ),
  Student(
    ID: 6,
    house: 'Gryffindor',
    likes: 'Heheh',
    name: 'Harry Potter',
    age: 25,
    course: 'IDE',
    yearLevel: 4,
    hobby: 'Quidditch',
    images: 'assets/SMC.jpg',
  ),
];
```

//App is now alphabetically arranged by name.



Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/student-list`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
oraiz@LAPTOP-S0MNC04S MINGW64 ~/Desktop/ulayjowonbkup/New folder/Gayl/FLUTTER/IT314AB_ORAIZ (feature/student-list)
$ git commit -m "Feat: Completed the objectives for CTF 4"
[feature/student-list b2c1f61] Feat: Completed the objectives for CTF 4
595 files changed, 8563 insertions(+), 414 deletions(-)
```